

Java Backend Architecture

Interview Series

Chapter 9.9

Cache Strategies

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 9.9 – Cache Strategies

Introduction

Selecting the right caching strategy is as important as selecting the cache technology. The wrong strategy leads to stale data, cache pollution, data loss, or unnecessary complexity. This chapter examines six fundamental strategies: **Cache-Aside** (lazy loading), **Read-Through**, **Write-Through**, **Write-Behind** (write-back), **Write-Around**, and **Refresh-Ahead**. Each strategy defines when and how the cache is populated (on read or write), how it handles misses, and what consistency guarantees it provides with respect to the backing store.

Core Concepts & Definitions

Cache-Aside	App explicitly checks cache on read; on miss, loads from DB and populates cache. Most common pattern.
Read-Through	App reads from cache only; cache intercepts misses and loads from DB transparently.
Write-Through	Write to cache AND DB synchronously; cache always consistent with DB.
Write-Behind	Write to cache immediately; flush to DB asynchronously in batch. Also called write-back.
Write-Around	Write directly to DB, bypassing cache; cache is populated lazily on subsequent reads.
Refresh-Ahead	Proactively reload cache entries predicted to expire before next access — zero miss latency.
Cache Warming	Pre-populating cache before traffic hits it to avoid cold-start miss storm.
Cache Pollution	Caching data that is never (or rarely) re-read; wastes cache space; hurts hit rate.
Write Amplification	Writing the same data to multiple tiers simultaneously; increases write latency.
Dirty Entry	A cache entry that has been modified but not yet flushed to the backing store (write-behind).
Cache Coherence	Ensuring all cache nodes (in a cluster) agree on the current value of a cached entry.
TTL (Time-to-Live)	How long a cache entry remains valid; after expiry it must be reloaded or revalidated.
Lazy Loading	Synonym for cache-aside; data loaded into cache only on first access.
Eager Loading	Pre-loading cache with predicted data at startup or before TTL expiry.
Write Coalescing	Write-behind optimization: multiple writes to the same key are merged before flushing.

Read Strategies: Cache-Aside vs Read-Through

Read Strategies: Cache-Aside vs Read-Through

Cache-Aside (Lazy Loading)

1. App checks cache
2. MISS → App queries DB
3. App writes to cache
4. App returns data

Pros: Only cache what is needed
Resilient to cache failure
Cache schema != DB schema OK

Cons: Cache miss penalty
Stale data possible
Implementation in app code

Read-Through

1. App reads from cache
2. Cache checks itself
3. MISS: cache loads DB
4. Cache stores + returns

Pros: App always hits cache
Cache handles loading logic
Good for Hazelcast MapStore

Cons: Cache must know schema
First read always slow
Provider-dependent feature

Write Strategies: Write-Through, Write-Behind, Write-Around, Refresh-Ahead

Write Strategies: Write-Through, Write-Behind, Write-Around

Write-Through

App writes cache + DB synchronously

Pros: cache always fresh

Cons: write latency 2x
Write amplification
Cache all written items

Write-Behind

App writes cache only; async flush to DB

Pros: low write latency
Batch DB writes

Cons: data loss risk
Complex recovery

Write-Around

App writes directly to DB bypassing cache

Pros: no cache pollution
Good for write-once data

Cons: first read misses
Cache must be primed

Refresh-Ahead (Predictive)

Background thread reloads entries predicted to expire soon
No cache miss penalty for hot entries; stale window is brief
Requires access-pattern prediction (LRU freq threshold)
Caffeine refreshAfterWrite implements this natively

Pros: zero latency on hot keys Cons: wastes resources on cold entries

Strategy Decision Matrix

Cache Strategy Decision Matrix

Strategy	Read Latency	Write Latency	Consistency	Complexity	Best For
Cache-Aside	Miss penalty	DB only	Eventual	Low	General OLTP
Read-Through	Miss penalty	DB only	Eventual	Medium	ORM/MapStore
Write-Through	Low (warm)	2x DB	Strong	Medium	Low-write reads
Write-Behind	Low (warm)	Very low	Eventual	High	High-write IoT
Write-Around	Miss penalty	DB only	Eventual	Low	Write-once files
Refresh-Ahead	Zero (warm)	DB + async	Near-real	Medium	Hot predictable

Choose based on read/write ratio, consistency requirements, and infrastructure constraints

Cache-Aside Pattern Implementation

```
// Manual Cache-Aside with Caffeine
@Service
public class ProductService {
    private final Cache<Long, Product> cache;
    private final ProductRepository repo;

    public ProductService(ProductRepository repo) {
        this.repo = repo;
        this.cache = Caffeine.newBuilder()
            .maximumSize(10_000)
            .expireAfterWrite(Duration.ofMinutes(10))
            .build();
    }

    public Product findById(Long id) {
        // Step 1: Check cache
        Product cached = cache.getIfPresent(id);
        if (cached != null) return cached; // HIT

        // Step 2: Load from DB (MISS)
        Product fromDb = repo.findById(id).orElseThrow();

        // Step 3: Populate cache
        cache.put(id, fromDb);

        return fromDb;
    }

    public Product update(Product product) {
        Product saved = repo.save(product);
        cache.put(saved.getId(), saved); // keep cache current
        return saved;
    }
}

// Spring shorthand - @Cacheable is cache-aside
@Cacheable(value = "products", key = "#id")
public Product findById(Long id) {
    return repo.findById(id).orElseThrow();
}
```

Read-Through Pattern (Caffeine LoadingCache)

```
// Caffeine LoadingCache = Read-Through
// Cache automatically loads from DB on miss - app never calls repo directly
LoadingCache<Long, Product> loadingCache = Caffeine.newBuilder()
    .maximumSize(10_000)
    .expireAfterWrite(Duration.ofMinutes(10))
    .build(id -> repo.findById(id));

// CacheLoader - called on MISS
Product p = loadingCache.get(42L); // auto-loads if absent

// Async read-through - non-blocking
AsyncLoadingCache<Long, Product> asyncCache = Caffeine.newBuilder()
    .maximumSize(10_000)
    .buildAsync(id -> CompletableFuture.supplyAsync(() -> repo.findById(id).orElseThrow()));

CompletableFuture<Product> cf = asyncCache.get(42L);

// Hazelcast MapStore = Read-Through at distributed level
// load() called by Hazelcast on IMap miss
public class ProductMapStore implements MapLoader<Long, Product> {
    public Product load(Long key) {
        return repo.findById(key).orElse(null);
    }
}
```

Write-Through Pattern

```
// Write-Through: write to cache AND DB in same transaction
@Service
@Transactional
public class ProductService {
    @CachePut(value = "products", key = "#product.id")
    public Product save(Product product) {
        // DB write + cache update happen together
        // @CachePut always executes method AND stores result
        return repo.save(product);
    }

    @CacheEvict(value = "products", key = "#id")
    public void delete(Long id) {
        repo.deleteById(id);
    }
}

// Manual write-through with Redis
public Product save(Product product) {
    Product saved = repo.save(product);
    // DB first
    redisTemplate.opsForValue().set("product:" + saved.getId(), saved, Duration.ofMinutes(30));
    // cache update
    return saved;
}
```

Write-Behind Pattern (Async DB Flush)

```
// Write-Behind: write to cache immediately, flush to DB asynchronously @Service public class
WriteBehindProductService { private final Cache<Long, Product> writeBuffer; private final
ProductRepository repo; public WriteBehindProductService(ProductRepository repo) { this.repo =
repo; this.writeBuffer = Caffeine.newBuilder() .maximumSize(50_000)
.removalListener(this::onEviction) .build(); } public void save(Product product) {
writeBuffer.put(product.getId(), product); // instant return } // Background flush every 5
seconds @Scheduled(fixedDelay = 5000) public void flush() { Map<Long, Product> dirty =
writeBuffer.asMap(); if (!dirty.isEmpty()) { repo.saveAll(dirty.values());
writeBuffer.invalidateAll(dirty.keySet()); } } private void onEviction(Long key, Product val,
RemovalCause cause) { if (cause == RemovalCause.SIZE || cause == RemovalCause.EXPIRED) {
repo.save(val); // flush on eviction too } } } // Hazelcast write-behind via
MapStore.writeDelay > 0 mapStoreConfig.setWriteDelaySeconds(5); // buffer 5s before DB flush
mapStoreConfig.setWriteBatchSize(200); // batch 200 entries per flush
```

Write-Around Pattern

```
// Write-Around: write directly to DB; cache NOT populated on write // Cache populated lazily
on first read (cache-aside) @Service public class FileMetadataService { @Autowired private
FileMetadataRepo repo; // Write-Around: no cache on write public FileMetadata
upload(FileMetadata meta) { return repo.save(meta); // DB only; cache not touched } //
Cache-Aside on read: lazily populated @Cacheable(value = "file-meta", key = "#id") public
FileMetadata findById(Long id) { return repo.findById(id).orElseThrow(); } } // Use Case: large
files uploaded once and read many times // Write-around prevents "write pollution": cache not
filled with // data that may never be read again (e.g., bulk import)
```

Refresh-Ahead Pattern

```
// Caffeine refreshAfterWrite = Refresh-Ahead // Stale entry served while background reload
runs LoadingCache<String, Config> configCache = Caffeine.newBuilder() .maximumSize(100)
.expireAfterWrite(Duration.ofMinutes(10)) // hard expiry
.refreshAfterWrite(Duration.ofMinutes(8)) // background refresh at 8min .build(key ->
configRepo.findById(key)); // async refresh // Access pattern: // 0-8 min: fresh entry served
directly // 8-10 min: stale entry served; background refresh triggered once // > 10 min: entry
expired; next access blocks to reload // Custom async executor for refresh LoadingCache<Long,
Product> productCache = Caffeine.newBuilder() .maximumSize(10_000)
.refreshAfterWrite(Duration.ofMinutes(5)) .executor(Executors.newFixedThreadPool(4)) //
dedicated refresh pool .build(id -> productRepo.findById(id).orElse(null)); // Manual
Refresh-Ahead with scheduled reload @Scheduled(fixedDelay = 60_000) public void
refreshHotEntries() { List<Long> hotKeys = accessCounters.topN(100); hotKeys.forEach(id -> {
Product fresh = productRepo.findById(id).orElse(null); if (fresh != null) cache.put(id, fresh);
}); }
```

Combining Strategies: L1/L2 with Write-Through

```
// L1 Caffeine (cache-aside) + L2 Redis (write-through) @Service public class
TieredCacheService { private final Cache<Long, Product> l1; // Caffeine private final
ValueOperations<String, Product> l2; // Redis public Product get(Long id) { // L1 check Product
p = l1.getIfPresent(id); if (p != null) return p; // L1 HIT // L2 check p = l2.get("product:" +
id); if (p != null) { l1.put(id, p); // back-fill L1 return p; // L2 HIT } // Both MISS: load
from DB p = productRepo.findById(id).orElseThrow(); l2.set("product:" + id, p,
Duration.ofMinutes(30)); l1.put(id, p); return p; } public Product save(Product product) {
Product saved = productRepo.save(product); // Write-Through l2.set("product:" + saved.getId(),
saved, Duration.ofMinutes(30)); l1.put(saved.getId(), saved); return saved; } }
```

Cache Warming Strategies

```
// 1. Eager warm-up on application start @Component public class CacheWarmup implements
ApplicationListener<ApplicationReadyEvent> { @Autowired private LoadingCache<Long, Product>
cache; @Autowired private ProductRepository repo; @Override public void
onApplicationEvent(ApplicationReadyEvent event) { log.info("Warming product cache...");
repo.findTop1000ByOrderByViewCountDesc() // top 1000 hot items .forEach(p ->
cache.put(p.getId(), p)); log.info("Cache warmed: {} entries", cache.estimatedSize()); } } //
2. Async background warm-up (non-blocking) @Async public void warmCacheAsync() { List<Long>
hotIds = analyticsService.getTopProductIds(500); hotIds.parallelStream().forEach(id -> {
cache.get(id); // triggers LoadingCache load }); } // 3. Redis pipeline for bulk cache seeding
redisTemplate.executePipelined((RedisCallback<Object>) conn -> { repo.findAll().forEach(p ->
conn.setEx(("product:" + p.getId()).getBytes(), 1800, serialize(p)); return null; }); }
```

Interview Questions & Answers

Q1. What is cache-aside and why is it the most common caching pattern?

Cache-aside (lazy loading): app checks cache first; on miss, loads from DB, stores in cache. Most common because: only caches data actually requested (no pollution), resilient to cache failures (app falls back to DB), works with any cache provider, and is easy to implement with @Cacheable.

Q2. What is the difference between cache-aside and read-through?

Cache-aside: app contains cache logic — it checks cache, loads from DB on miss, puts result in cache. Read-through: cache itself contains the loading logic — app only reads from cache; cache transparently loads from DB on miss. Read-through centralizes loading logic; requires cache to know DB schema.

Q3. What is write-through caching?

Write-through: on every write, data is written to cache AND DB synchronously before returning. Cache is always consistent with DB. Trade-off: write latency doubles (2 writes per operation), write amplification: every written entry is cached, even if never read again.

Q4. What is write-behind (write-back) caching?

Write-behind: writes go to cache only; DB is updated asynchronously in background (batch flush). Pros: write latency is sub-millisecond (only cache write). Batch DB writes are efficient. Cons: risk of data loss if cache fails before flush; complex recovery; dirty entries inconsistency.

Q5. When would you use write-around caching?

When data is written once and rarely or never read back through the cache (bulk imports, log data, large media files). Writing to cache would pollute it with entries that won't be re-read, evicting hot entries and hurting hit rate. Write-around prevents this cache pollution.

Q6. What is refresh-ahead caching?

Refresh-ahead proactively reloads cache entries before their TTL expires — based on access prediction. Caffeine's refreshAfterWrite implements this: serves stale entry while background reload runs. Zero miss latency for hot entries. Trade-off: wastes DB reads for cold entries that expire without reuse.

Q7. What is cache warming and why does it matter?

Cache warming pre-populates the cache before traffic arrives. Without it, a cold cache on startup causes a miss storm: all requests hit DB simultaneously (thundering herd). Warmup strategies: ApplicationReadyEvent listener, async background job, export/import from persistent store.

Q8. What is the write-through vs write-behind consistency trade-off?

Write-through: strong consistency (cache = DB always), but higher write latency and write amplification. Write-behind: lower write latency and better write throughput, but eventual consistency (DB may lag) and risk of dirty-entry data loss on cache failure.

Q9. How does Caffeine implement refresh-ahead?

`refreshAfterWrite(duration)`: after the specified duration since last load, the next read returns the stale value immediately AND triggers an asynchronous reload via the `CacheLoader`. The fresh value is stored when the async load completes. `expireAfterWrite` is a hard expiry; `refreshAfterWrite` is soft.

Q10. What is cache pollution and how do you prevent it?

Cache pollution: caching data that is never re-read, wasting capacity and evicting hot entries. Prevention: write-around for write-once data, conditional caching (`@Cacheable` condition), size-based eviction (W-TinyLFU frequency filter prevents one-hit wonders from evicting hot entries), separate caches for different data access patterns.

Q11. How do you implement write-behind in a Spring Boot application?

Pattern: write to in-memory map/cache; `@Scheduled` task flushes dirty entries to DB periodically. Or use Hazelcast MapStore with `writeDelay > 0`. Or use Redis Streams: write to Redis stream; consumer group reads and persists to DB. For durability, Redis AOF ensures stream persists before flush.

Q12. What is the hot-cold data problem in caching?

Hot data: frequently accessed (top 1% of keys account for 80% of requests). Cold data: rarely accessed. Cache should hold hot data. W-TinyLFU (Caffeine) handles this well — frequency filter admits hot, rejects cold. For write-through, cold data pollutes cache; combine with write-around for cold paths.

Q13. What is double-checked locking in cache-aside?

Pattern: check cache, lock, check cache again (double-check), load from DB if still absent. Prevents multiple threads loading the same key simultaneously. Java: `synchronized` block or `ConcurrentHashMap.computeIfAbsent()`. Caffeine `LoadingCache` provides this automatically per key.

Q14. How do you handle cache and database inconsistency in write-through?

Write-through writes cache first, then DB. If DB write fails, cache has stale data. Solution: write DB first, then update cache (less risky). On DB failure, evict cache entry rather than leave stale. Or use `@Transactional` + `TransactionSynchronizationManager` to defer cache write to after DB commit.

Q15. What is the stale-read problem in cache-aside?

Between cache population and TTL expiry, the DB may have been updated by another service or direct DB write. Cache serves stale data. Solutions: short TTL, event-driven invalidation (Kafka/CDC evicts cache on DB change), versioned keys (include version in cache key; new version = new key = automatic miss).

Q16. What is a write-coalescing optimization in write-behind?

If the same key is written multiple times while dirty (before flush), write-behind stores only the latest value and performs a single DB write. Reduces DB write amplification significantly for high-update scenarios (e.g., counters, sensor readings). Hazelcast MapStore implements this via `writeBatchSize`: combines multiple updates to same key into one `store()` call.

Q17. What is the read-your-writes consistency challenge with cache-aside?

After a write (update DB, evict cache), a subsequent read by the same user should see the updated data. If the read hits a replica DB that hasn't replicated the write yet, it caches stale data. Solution: route reads immediately post-write to primary DB, or add a short TTL to read-your-writes guarantee window, or use version-stamped cache keys.

Q18. Describe refresh-ahead vs stale-while-revalidate.

Refresh-ahead (server): cache proactively reloads before TTL, based on access frequency prediction. stale-while-revalidate (HTTP): serve stale while revalidating with origin in background. Both serve stale to avoid miss latency. Refresh-ahead is triggered by access pattern; stale-while-revalidate is triggered by TTL window. Both reduce latency spikes.

Q19. How do you implement multi-key cache population efficiently?

getAll(keys): Caffeine LoadingCache.getAll(Set) calls CacheLoader.loadAll() for missing keys. loadAll() should batch-load from DB (one SELECT ... WHERE id IN (...)). Never call get() in a loop for multiple keys — each miss triggers individual DB call. Spring: no built-in multi-key @Cacheable; implement manually with cache.getAll().

Q20. What is a cache stampede in the context of write-through?

Cache stampede is primarily a read-pattern problem. Write-through actually mitigates stampede: because write populates cache, subsequent reads always hit. The write-through stampede occurs at startup before any writes happen — the cache is empty and reads miss. Mitigate with warmup.

Q21. What is the fan-out problem in cache invalidation?

When one entity is cached in many different cache entries (products in category caches, search results, recommendation lists), a single product update requires invalidating all related caches. Solutions: fine-grained invalidation by tag (Surrogate-Key), generation counters in cache keys, short TTLs for derived/composite caches.

Q22. How does write-around interact with a read-through cache?

Write-around bypasses cache on write; cache is populated lazily on first read. In read-through, the miss triggers a DB load and cache population. So write-around + read-through = data is cached on first read after write. Each write generates exactly one subsequent cache miss per key before the entry is cached. This combination is suitable for write-once, read-many data.

Q23. What is the JCache (JSR-107) approach to cache strategies?

JCache defines CacheEntryProcessor for atomic cache operations and ExpiryPolicy for TTL strategies (CreatedExpiryPolicy, AccessedExpiryPolicy, ModifiedExpiryPolicy, TouchedExpiryPolicy, EternalExpiryPolicy). Write-through and read-through are supported via CacheWriter and CacheLoader SPI. Both Ehcache and Hazelcast implement JSR-107.

Q24. What happens to write-behind entries on application shutdown?

Dirty entries not yet flushed to DB are lost unless explicitly handled. Solutions: register JVM shutdown hook to flush buffer on SIGTERM, use Caffeine's removalListener to persist on eviction/shutdown, store write-behind queue in Redis (durable) not just in-memory, use database WAL/journal as additional durability net.

Q25. Explain the cache invalidation bus pattern.

Microservices publish invalidation events (e.g., ProductUpdated{id:42}) to a message broker (Kafka, Redis Pub/Sub). All service instances subscribe and evict local cache entries. Write-through: publish after DB write. Write-behind: publish when cache is updated. Provides cluster-wide cache consistency without direct coupling between services.

Q26. When is write-through harmful?

Write-through is harmful when: 1) Write-to-read ratio is high (many writes, few reads) — data cached but never re-read (pollution). 2) Write latency is critical — 2x write cost is unacceptable. 3) Cache is too small relative to write volume — constant eviction defeats caching. Use write-around for write-heavy, write-through only for read-heavy after initial writes.

Q27. How do you choose between `refreshAfterWrite` and `expireAfterWrite` in Caffeine?

`expireAfterWrite`: hard TTL — entry removed after N time; next access blocks to reload. Use when stale data is unacceptable. `refreshAfterWrite`: soft TTL — entry stays (stale) until reloaded; next access returns stale immediately + triggers async reload. Use for hot keys where latency spikes are worse than brief staleness. Always set `expireAfterWrite` > `refreshAfterWrite`.

Q28. What is the N-tier caching strategy?

Multiple levels: L1 (JVM/Caffeine, nanoseconds, small), L2 (Redis, milliseconds, large), L3 (CDN/Varnish, 10ms, global). Each level has different capacity, latency, and TTL. On read: check L1 → L2 → L3 → origin. On write (write-through): update L1 + L2 + L3 or use short TTLs at L1/L2 + CDN surrogate-key for L3. Match tier to data volatility.

Q29. What is cache key design best practice?

Key design: include entity type (prevents collisions), entity identifier, version/scope if multi-tenant. Format: `{entity}:{id}` or `{tenant}:{entity}:{id}`. Avoid non-deterministic keys (random, timestamp). Hash long keys. Be consistent — key generation logic must match between write and read paths. Test key generation with unit tests.

Q30. How does the Circuit Breaker pattern interact with cache strategies?

When cache provider (Redis) is unavailable, circuit breaker opens, bypassing cache calls entirely. Read strategy: cache-aside fallback to DB. Write strategy: skip cache write (write-around fallback). When circuit closes, cache is cold — consider warmup. Caffeine (in-process) doesn't need circuit breaker — it's always available. Redis does.